

CORE CONCEPTS

How Claude Code works

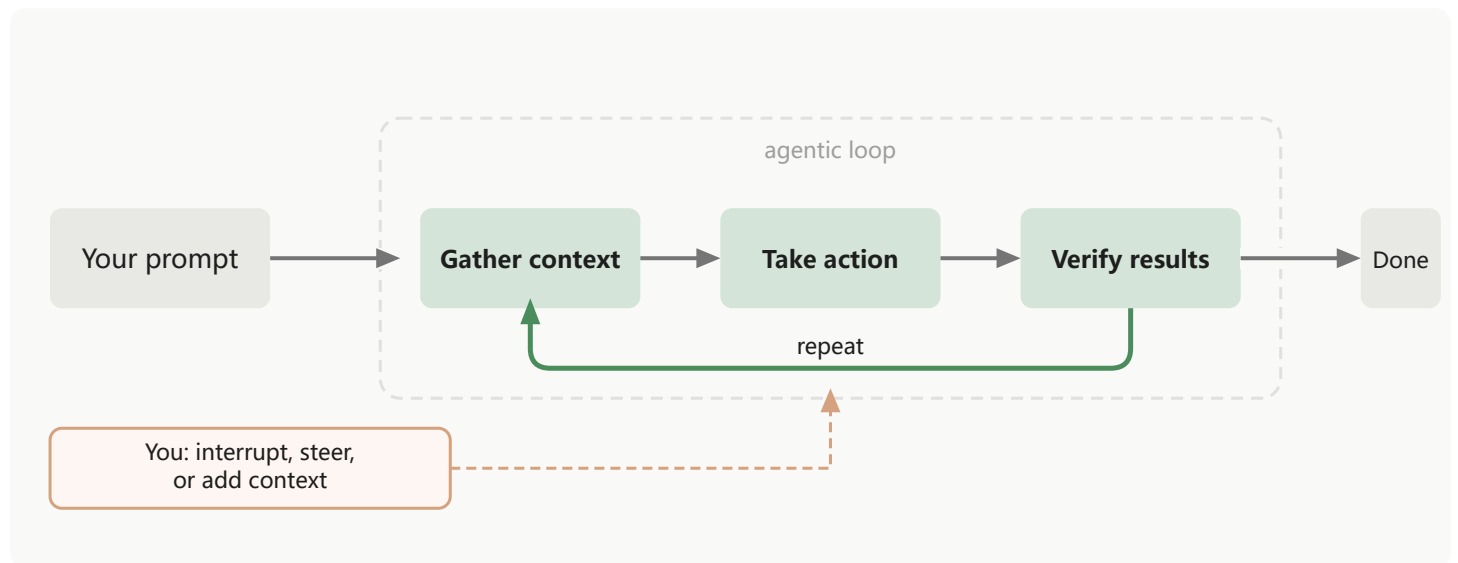
Understand the agentic loop, built-in tools, and how Claude Code interacts with your project.

Claude Code is an agentic assistant that runs in your terminal. While it excels at coding, it can help with anything you can do from the command line: writing docs, running builds, searching files, researching topics, and more.

This guide covers the core architecture, built-in capabilities, and [tips for working effectively](#). For step-by-step walkthroughs, see [Common workflows](#). For extensibility features like skills, MCP, and hooks, see [Extend Claude Code](#).

The agentic loop

When you give Claude a task, it works through three phases: **gather context**, **take action**, and **verify results**. These phases blend together. Claude uses tools throughout, whether searching files to understand your code, editing to make changes, or running tests to check its work.



The loop adapts to what you ask. A question about your codebase might only need context gathering. A bug fix cycles through all three phases repeatedly. A refactor might involve extensive verification. Claude decides what each step requires based on what it learned from the previous step, chaining dozens of actions together and course-correcting along the way.

You’re part of this loop too. You can interrupt at any point to steer Claude in a different direction, provide additional context, or ask it to try a different approach. Claude works autonomously but stays responsive to your input.

The agentic loop is powered by two components: models that reason and tools that act. Claude Code serves as the **agentic harness** around Claude: it provides the tools, context management, and execution environment that turn a language model into a capable coding agent.

Models

Claude Code uses Claude models to understand your code and reason about tasks. Claude can read code in any language, understand how components connect, and figure out what needs to change to accomplish your goal. For complex tasks, it breaks work into steps, executes them, and adjusts based on what it learns.

Multiple models are available with different tradeoffs. Sonnet handles most coding tasks well. Opus provides stronger reasoning for complex architectural decisions. Switch with `/model` during a session or start with `claude --model <name>` .

When this guide says “Claude chooses” or “Claude decides,” it’s the model doing the reasoning.

Tools

Tools are what make Claude Code agentic. Without tools, Claude can only respond with text. With tools, Claude can act: read your code, edit files, run commands, search the web, and interact with external services. Each tool use returns information that feeds back into the loop, informing Claude’s next decision.

The built-in tools generally fall into five categories, each representing a different kind of agency.

Category	What Claude can do
File operations	Read files, edit code, create new files, rename and reorganize
Search	Find files by pattern, search content with regex, explore codebases
Execution	Run shell commands, start servers, run tests, use git
Web	Search the web, fetch documentation, look up error messages
Code intelligence	See type errors and warnings after edits, jump to definitions, find references (requires <u>code intelligence plugins</u>)

These are the primary capabilities. Claude also has tools for spawning subagents, asking you questions, and other orchestration tasks. See [Tools available to Claude](#) for the complete list.

Claude chooses which tools to use based on your prompt and what it learns along the way. When you say “fix the failing tests,” Claude might:

1. Run the test suite to see what’s failing
2. Read the error output
3. Search for the relevant source files
4. Read those files to understand the code
5. Edit the files to fix the issue
6. Run the tests again to verify

Each tool use gives Claude new information that informs the next step. This is the agentic loop in action.

Extending the base capabilities: The built-in tools are the foundation. You can extend what Claude knows with [skills](#), connect to external services with [MCP](#), automate workflows with [hooks](#), and offload tasks to [subagents](#). These extensions form a layer on top of the core agentic loop. See [Extend Claude Code](#) for guidance on choosing the right extension for your needs.

What Claude can access

This guide focuses on the terminal. Claude Code also runs in [VS Code](#), [JetBrains IDEs](#), and other environments.

When you run `claude` in a directory, Claude Code gains access to:

- **Your project.** Files in your directory and subdirectories, plus files elsewhere with your permission.
- **Your terminal.** Any command you could run: build tools, git, package managers, system utilities, scripts. If you can do it from the command line, Claude can too.
- **Your git state.** Current branch, uncommitted changes, and recent commit history.
- **Your [CLAUDE.md](#).** A markdown file where you store project-specific instructions, conventions, and context that Claude should know every session.
- **[Auto memory](#).** Learnings Claude saves automatically as you work, like project patterns and your preferences. The first 200 lines or 25KB of `MEMORY.md`, whichever comes first, load at the start

of each session.

- **Extensions you configure.** MCP servers for external services, skills for workflows, subagents for delegated work, and Claude in Chrome for browser interaction.

Because Claude sees your whole project, it can work across it. When you ask Claude to “fix the authentication bug,” it searches for relevant files, reads multiple files to understand context, makes coordinated edits across them, runs tests to verify the fix, and commits the changes if you ask. This is different from inline code assistants that only see the current file.

Environments and interfaces

The agentic loop, tools, and capabilities described above are the same everywhere you use Claude Code. What changes is where the code executes and how you interact with it.

Execution environments

Claude Code runs in three environments, each with different tradeoffs for where your code executes.

Environment	Where code runs	Use case
Local	Your machine	Default. Full access to your files, tools, and environment
Cloud	Anthropic-managed VMs	Offload tasks, work on repos you don't have locally
Remote Control	Your machine, controlled from a browser	Use the web UI while keeping everything local

Interfaces

You can access Claude Code through the terminal, the desktop app, IDE extensions, claude.ai/code, Remote Control, Slack, and CI/CD pipelines. The interface determines how you see and interact with Claude, but the underlying agentic loop is identical. See Use Claude Code everywhere for the full list.

Work with sessions

Claude Code saves your conversation locally as you work. Each message, tool use, and result is

written to a plaintext JSONL file under `~/.claude/projects/` , which enables rewinding, resuming, and forking sessions. Before Claude makes code changes, it also snapshots the affected files so you can revert if needed. For paths, retention, and how to clear this data, see application data in `~/.claude` .

Sessions are independent. Each new session starts with a fresh context window, without the conversation history from previous sessions. Claude can persist learnings across sessions using auto memory, and you can add your own persistent instructions in CLAUDE.md.

Work across branches

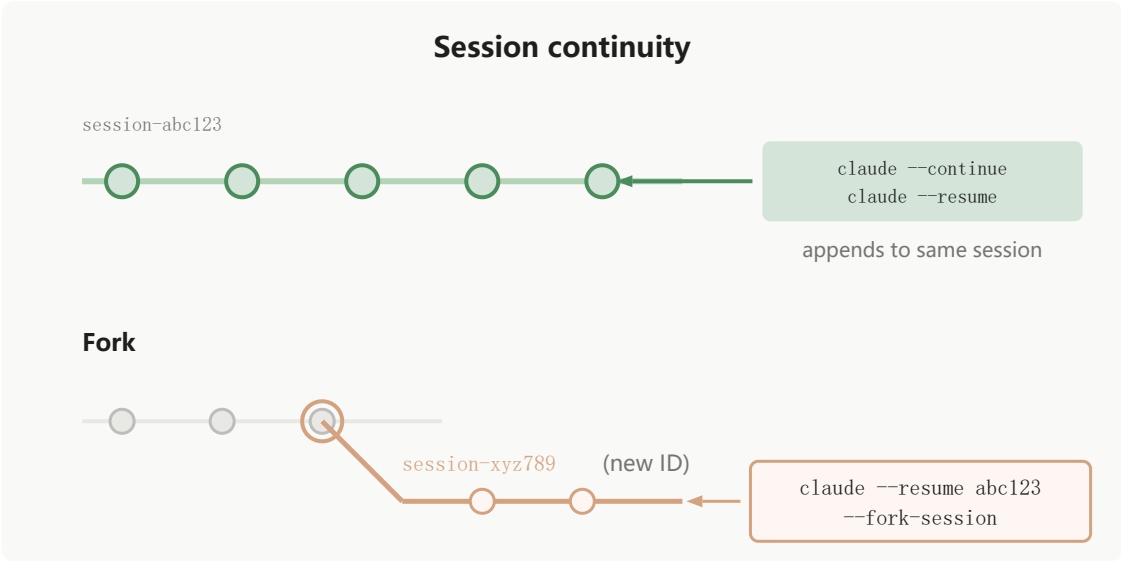
Each Claude Code conversation is a session tied to your current directory. The `/resume` picker shows sessions from the current worktree by default, with keyboard shortcuts to widen the list to other worktrees or projects. See Manage sessions for the full list of picker shortcuts and how name resolution works.

Claude sees your current branch's files. When you switch branches, Claude sees the new branch's files, but your conversation history stays the same. Claude remembers what you discussed even after switching.

Since sessions are tied to directories, you can run parallel Claude sessions by using git worktrees, which create separate directories for individual branches.

Resume or fork sessions

Resuming a session with `claude --continue` or `claude --resume` reopens it under the same session ID and appends new messages to the existing conversation. Forking with `--fork-session` or `/branch` copies the history into a new session ID, leaving the original unchanged.



For the resume flags, the `/resume` picker, naming, and what happens when the same session is open in two terminals, see [Manage sessions](#).

The context window

Claude's context window holds your conversation history, file contents, command outputs, **CLAUDE.md**, **auto memory**, loaded skills, and system instructions. As you work, context fills up. Claude compacts automatically, but instructions from early in the conversation can get lost. Put persistent rules in CLAUDE.md, and run `/context` to see what's using space.

For an interactive walkthrough of what loads and when, see [Explore the context window](#).

When context fills up

Claude Code manages context automatically as you approach the limit. It clears older tool outputs first, then summarizes the conversation if needed. Your requests and key code snippets are preserved; detailed instructions from early in the conversation may be lost. Put persistent rules in CLAUDE.md rather than relying on conversation history.

To control what's preserved during compaction, add a "Compact Instructions" section to CLAUDE.md or run `/compact` with a focus (like `/compact focus on the API changes`).

If a single file or tool output is so large that context refills immediately after each summary, Claude Code stops auto-compacting after a few attempts and shows an error instead of looping. See [Auto-compaction stops with a thrashing error](#) for recovery steps.

Run `/context` to see what's using space. MCP tool definitions are deferred by default and loaded on demand via **tool search**, so only tool names consume context until Claude uses a specific tool. Run `/mcp` to check per-server costs.

Manage context with skills and subagents

Beyond compaction, you can use other features to control what loads into context.

Skills load on demand. Claude sees skill descriptions at session start, but the full content only loads when a skill is used. For skills you invoke manually, set `disable-model-invocation: true` to keep descriptions out of context until you need them. For skills you didn't write, use [skill overrides](#) to do the same from settings.

Subagents get their own fresh context, completely separate from your main conversation. Their work doesn't bloat your context. When done, they return a summary. This isolation is why subagents

help with long sessions.

See [context costs](#) for what each feature costs, and [reduce token usage](#) for tips on managing context.

Stay safe with checkpoints and permissions

Claude has two safety mechanisms: checkpoints let you undo file changes, and permissions control what Claude can do without asking.

Undo changes with checkpoints

Every file edit is reversible. Before Claude edits any file, it snapshots the current contents. If something goes wrong, press `Esc` twice to rewind to a previous state, or ask Claude to undo.

Checkpoints are local to your session, separate from git. They only cover file changes. Actions that affect remote systems (databases, APIs, deployments) can't be checkpointed, which is why Claude asks before running commands with external side effects.

Control what Claude can do

Press `Shift+Tab` to cycle through permission modes:

- **Default:** Claude asks before file edits and shell commands
- **Auto-accept edits:** Claude edits files and runs common filesystem commands like `mkdir` and `mv` without asking, still asks for other commands
- **Plan mode:** Claude explores and proposes a plan without editing your source files; permission prompts still apply as in default mode
- **Auto mode:** Claude evaluates all actions with background safety checks. Currently a research preview

You can also allow specific commands in `.claude/settings.json` so Claude doesn't ask each time. This is useful for trusted commands like `npm test` or `git status`. Settings can be scoped from organization-wide policies down to personal preferences. See [Permissions](#) for details.

Work effectively with Claude Code

These tips help you get better results from Claude Code.

Ask Claude Code for help

Claude Code can teach you how to use it. Ask questions like “how do I set up hooks?” or “what’s the best way to structure my CLAUDE.md?” and Claude will explain.

Built-in commands also guide you through setup:

- `/init` walks you through creating a CLAUDE.md for your project
- `/agents` helps you configure custom subagents
- `/doctor` diagnoses common issues with your installation

It’s a conversation

Claude Code is conversational. You don’t need perfect prompts. Start with what you want, then refine:

Fix the login bug

[Claude investigates, tries something]

That's not quite right. The issue is in the session handling.

[Claude adjusts approach]

When the first attempt isn’t right, you don’t start over. You iterate.

Interrupt and steer

You can redirect Claude at any point without waiting for the turn to finish or starting over:

- **Press** `Esc` to stop Claude immediately. The running tool call is canceled and Claude waits for your next instruction.
- **Type a correction and press** `Enter` to send it without stopping the running tool. Claude reads

it as soon as the current action completes and adjusts before deciding its next step.

Be specific upfront

The more precise your initial prompt, the fewer corrections you'll need. Reference specific files, mention constraints, and point to example patterns.

```
The checkout flow is broken for users with expired cards.  
Check src/payments/ for the issue, especially token refresh.  
Write a failing test first, then fix it.
```

Vague prompts work, but you'll spend more time steering. Specific prompts like the one above often succeed on the first attempt.

Give Claude something to verify against

Claude performs better when it can check its own work. Include test cases, paste screenshots of expected UI, or define the output you want.

```
Implement validateEmail. Test cases: 'user@example.com' → true,  
'invalid' → false, 'user@.com' → false. Run the tests after.
```

For visual work, paste a screenshot of the design and ask Claude to compare its implementation against it.

Explore before implementing

For complex problems, separate research from coding. Use plan mode (`Shift+Tab` twice) to analyze the codebase first:

```
Read src/auth/ and understand how we handle sessions.  
Then create a plan for adding OAuth support.
```

Review the plan, refine it through conversation, then let Claude implement. This two-phase approach produces better results than jumping straight to code.

Delegate, don't dictate

Think of delegating to a capable colleague. Give context and direction, then trust Claude to figure out the details:

```
The checkout flow is broken for users with expired cards.  
The relevant code is in src/payments/. Can you investigate and  
fix it?
```

You don't need to specify which files to read or what commands to run. Claude figures that out.

What's next



Extend with features

Add Skills, MCP connections, and custom commands



Common workflows

Step-by-step guides for typical tasks